SHELTER-SEEKER OVERVIEW

This project is basically a full-stack application, where the frontend and backend are tightly integrated to provide the complete user experience. The frontend is built to handle the user interface, routing, form handling, map-based property selection, image upload flow, and real-time chat interaction, while the main backend is built using Node.js, Express, MongoDB, and Mongoose, along with a separate fraud detection microservice. On the backend side, the Node server manages the core business logic such as authentication, users, listings, locality insights, trust management, admin moderation, uploads, and chat. Both frontend and backend communicate through APIs and socket connections, with cookie-based authentication, CORS with credentials, centralized error handling, third-party integrations like map/geocoding and image hosting, and Socket.IO attached to the same HTTP server for real-time messaging.

Starting with the auth module, a user can either sign up and sign in using email and password or use Google sign-in. In the normal sign-in flow, the password is hashed using bcrypt before storing it in the database, and during login it is compared securely. Once the user is authenticated, the backend creates a JWT containing the user ID and role, and stores that token in an HTTP-only cookie called access_token. That cookie is then used by middleware to protect routes, identify the current user, and also check whether the user is an admin. There is also logout support, and one more thing the auth flow does is check whether the user is banned before allowing login.

In the user module, I handle profile-level operations like updating user details, changing password, fetching a user profile, getting that user's own listings, and deleting the account. When a user is deleted, I am not just deleting the user document; I also clean up their listings and even their chat conversations, so the data stays consistent. I also maintain user roles such as buyer, seller, and admin, which are important because different modules use role-based authorization.

Coming to the listing module, this is one of the core modules of the project. The listing model stores all major property information such as property name, description, address, regular and discounted price, bedroom and washroom count, furnished and parking flags, offer status, property type, image URLs, owner reference, moderation status, fraud detection result, seller insight data, and location coordinates. When a seller creates a listing, the backend first validates the required fields. After that, it reads the coordinates selected from the OSM map picker, computes a geohash for spatial grouping, and then calls OpenStreetMap reverse geocoding to derive a readable locality name. That locality name and geohash are stored on the listing itself, so the listing becomes both searchable and locality-aware.

The image upload part is also important, and this is one of the places where the frontend and backend coordinate closely. The frontend does not directly save raw image files into MongoDB. Instead, when the user selects images, the client reads them as Data URLs and sends them to the backend upload endpoint. The backend validates that the payload is actually an image, signs the upload request using Cloudinary credentials, uploads the image to Cloudinary, and then returns the secure URL and public ID. Those Cloudinary URLs are what finally get stored inside the listing's imageUrls field. So the listing only stores media links, not the actual binary image data.

Another major part inside listing creation is fraud detection. Before or during listing persistence, the Node backend calls a separate fraud detection service running in Python Flask. That service loads a trained model and label encoder on startup and exposes endpoints like /detect-fraud and /health. From the Node side, I send a payload containing features such as price, estimated area, bedroom count, and city extracted from the address. The fraud service returns values like fraudScore, anomalyScore, isFraudulent, and feature details, and that result is stored inside the listing document. So fraud detection is not hardcoded in the main server; it is separated as a

microservice, and the Node backend consumes it over HTTP. I also have a health-check endpoint for that integration. On top of that, my listing search API is optimized using Redis caching, so repeated filtered searches do not always hit MongoDB directly.

The locality insights module is another important feature and a bit more advanced than normal CRUD. Here, the listing stores a localityName and seller-provided locality feedback, while a separate locality collection stores the aggregated neighborhood-level data. So I am not mainly using a direct MongoDB reference from listing to locality; the connection is driven by computed locality information. When a listing is created in a locality that does not yet exist in the database, the backend upserts a new locality document. If it already exists, it updates the same locality and increases the total listing count. Each listing can carry seller insight categories such as water supply, power supply, traffic, safety, schools, and daily needs, and each category can have both a rating and a description.

The locality score calculation is based on aggregation, not on a single listing. For each listing, the seller gives ratings for those locality-related categories. Then, for a given locality, the backend takes all listings that belong to that locality, converts the category labels like Excellent, Good, Average, Poor, and Very Poor into numeric scores, averages them category-wise, and then derives an overall locality score on a 10-point scale. So the final locality score represents the collective input of multiple property owners in the same neighborhood. On top of that, I also support locality-based Q&A. A locality can have many questions, and each question can have multiple answers from owners, residents, or visitors. When the frontend opens locality insights for a listing, the backend returns the seller insight for that specific listing, the aggregated locality data for the neighborhood, and the list of locality questions and answers.

For trust and admin, I have two related but different layers. One is seller-level trust, and the other is listing-level moderation. At the seller level, every user has trust points, seller stats, rating, trust history, and a verified-seller style badge when the score crosses the threshold. Some trust updates are automatic, such as when a seller creates a listing, and some are controlled by the admin. The trust service also supports fetching seller profiles, showing top sellers, viewing trust history, batch trust updates, and seller-level statistics. At the listing level, the admin can approve or reject properties, and once a listing is approved, that property gets the verified or approved trust signal visible to users. So one trust signal is about the seller's overall reputation, and the other is about whether a specific property listing has been moderated.

The admin module is basically the governance and moderation layer of the platform. Admin can view users with filtering and pagination, ban or unban users, change their roles, manage listings, approve or reject listings, delete listings if required, review user reports, and view dashboard analytics such as total users, pending listings, approved listings, rejected listings, banned users, and report counts. One important thing here is that I also log admin activities, so whenever an admin performs actions like banning a user, changing trust points, approving a listing, or resolving a report, that action is stored with metadata such as action type, target entity, IP address, and user agent. That creates an audit trail, which is useful for platform transparency.

Finally, the chat module combines REST APIs with Socket.IO for real-time communication. First, I create a normal HTTP server from the Express app, and then I attach a Socket.IO server to it. Socket.IO begins with an HTTP handshake and upgrades to WebSocket when possible, which is why the connection becomes persistent and full duplex. In my design, a conversation is identified by a conversationId built from the landlord ID and tenant ID, so the room is basically pair-based, while the listing ID is stored as metadata. There are normal REST endpoints to create or fetch a

conversation, start one from a listing, append a message, and list all conversations for the current user.

On the real-time side, when the socket connects, the client sends its userId in the handshake query, and the server can track active sockets in memory. Then both participants join a Socket.IO room using join_conversation. When a chat_message event is fired, the backend first upserts the conversation in MongoDB and pushes the new message into the messages array, and after persisting it, it broadcasts that message to everyone in the same room. There is also a typing event for typing indicators and a disconnect event for cleanup. So the simple way to explain it is that REST is used for bootstrapping and history, while Socket.IO is used for instant delivery of live chat events.

Overall, this project is not just a basic CRUD full-stack application. On the frontend side, it provides the complete user-facing experience such as authentication screens, property listing forms, map-based location picking, image upload handling, locality insight views, admin panels, and real-time chat interfaces, while on the backend it integrates authentication, role-based authorization, media upload through Cloudinary, geolocation and reverse geocoding through OpenStreetMap, locality intelligence, trust scoring, moderation workflows, report handling, Redis caching, real-time chat through Socket.IO, and a separate Flask-based fraud detection service. Since both layers are tightly connected through APIs and socket events, the platform becomes much more intelligent, responsive, and trustworthy than a normal property listing application.